

CLASE 5 · UNIDAD 2 · SEMANA 5

Fine-tuning eficiente

Del modelo base al asistente: instruction tuning, y cómo ajustarlo barato — PEFT, LoRA y QLoRA.

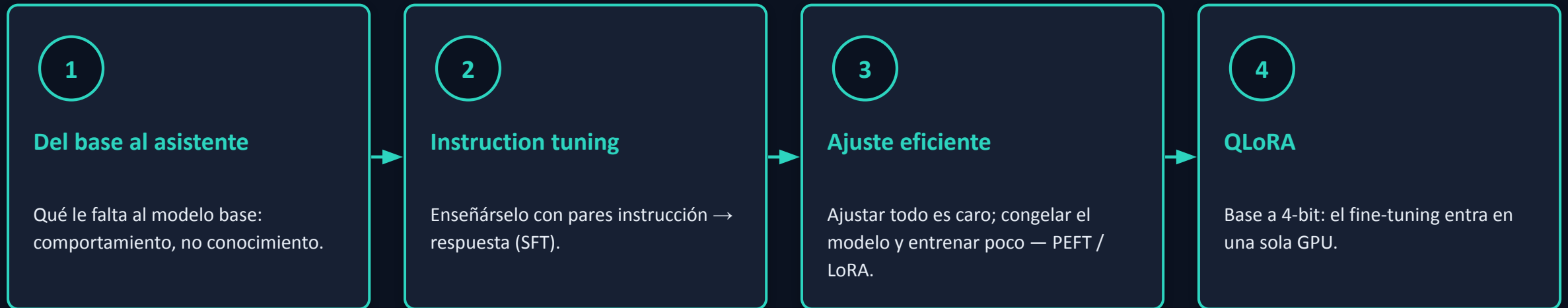
Inteligencia Artificial Generativa Avanzada y Sistemas Multi Agente

Licenciatura en Tecnología Digital · Universidad Torcuato Di Tella



El recorrido

Un modelo base sabe, pero no se comporta como asistente. Este recorrido lo convierte en uno — y lo hace barato. Cuatro tramos, un mismo hilo.



El hilo. la misma loss next-token de punta a punta. Lo único que cambia es con qué datos entrenás (demostraciones, no texto crudo) y qué dejás mover (cada vez menos pesos, cada vez más barato).

FINE-TUNING

Del base al asistente

Tiene el conocimiento; le falta el comportamiento. Cómo se enseña.



De dónde venimos, y qué falta

De dónde venimos. el preentrenamiento entregó un modelo base: un predictor de próximo token con competencia amplia. Lo caro ya está pago — sabe muchísimo del mundo.

Qué le falta. no se comporta como asistente. Ante «¿Cuál es la capital de Francia?», puede seguir con «¿Y la de Alemania? ¿Y la de Italia?»: completa una lista de preguntas en vez de responder.

Por qué lo hace. su único objetivo fue predecir el próximo token de documentos de internet, así que aprendió la distribución del texto de internet. Y ahí una pregunta suele ir seguida de más preguntas —un cuestionario, una FAQ—, no de su respuesta. Continúa el patrón: hace exactamente aquello para lo que se lo entrenó.

Dónde arrancamos. el problema no es el conocimiento, es el comportamiento. Todo el recorrido convierte ese «sabe» en «responde».

«¿Cuál es la capital de Francia?»

modelo base

«¿Y la de Alemania? ¿Y la de Italia?...»
sigue el patrón

lo que queremos «**París.**»
responde

en la web, una pregunta suele ir seguida de otra — no de su respuesta

No es ignorancia — es comportamiento. sabe que es París: el conocimiento está en los pesos. Prueba: con few-shot (unos ejemplos pregunta→respuesta en el prompt) el base ya responde. Falta que responder sea el default.

Enseñar comportamiento con ejemplos

Qué esperás de un asistente. tres cosas que un completador de texto no hace de fábrica: responder (no seguir el patrón), seguir instrucciones (hacer lo que le pedís) y darlo en un formato útil (conciso, en el registro adecuado).

La idea: mostrarle el comportamiento. el modelo aprende imitando texto. Entonces le damos ejemplos del comportamiento que queremos —demostraciones de «así se responde bien»— y lo entrenamos a imitarlas. Eso es supervised fine-tuning (SFT).

No es un algoritmo nuevo. es el mismo loop de next-token de siempre (forward → loss → backward → update). Lo único que cambia son los datos: en vez de texto crudo de la web, pares (instrucción → respuesta ideal) curados por humanos para demostrar buen comportamiento. El modelo clona esas demostraciones — behavior cloning.

El plan. enseñar comportamiento = entrenar al modelo a imitar demostraciones de buen comportamiento. Mismo loop; los datos son pares curados, no texto crudo.



Por qué alcanza con tan poca data

La desproporción. el pretraining costó billones de tokens, meses de cómputo y millones de dólares. El SFT usa miles o decenas de miles de ejemplos y corre en horas: tres o cuatro órdenes de magnitud menos.

Por qué alcanza. porque la competencia ya está adentro, comprada en el pretraining. El SFT no enseña de cero: dirige lo que ya está. No agrega conocimiento, corre el default hacia el comportamiento-respuesta. Es la contracara de que, con few-shot, el base ya respondía: la capacidad estaba, solo faltaba activarla.

La línea que no hay que cruzar. responder, seguir instrucciones y formato son todo comportamiento → se aprenden de demostraciones (SFT). Conocimiento nuevo —un idioma, un dominio— no es comportamiento: eso necesita continued pretraining. Un dataset chico de instrucciones no mete un idioma.

La regla.

Comportamiento nuevo → SFT (barato, poca data).

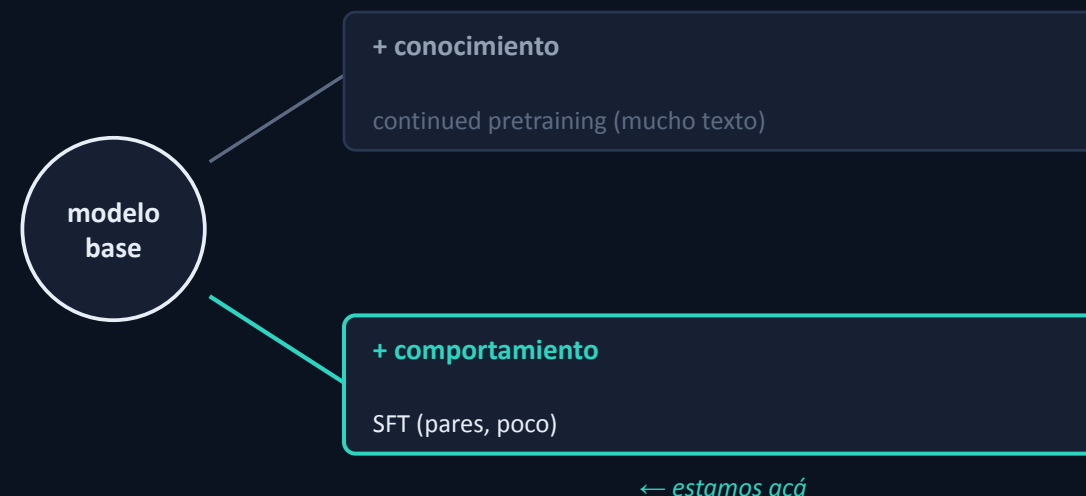
Conocimiento nuevo → continued pretraining. No los mezcles.

cuánta data hace falta

pretraining · ~billones de tokens

SFT · ~miles

≈ 10⁴× menos



FINE-TUNING

Instruction tuning

De completar texto a seguir instrucciones: enseñar el comportamiento con demostraciones.



De texto plano a pares instrucción → respuesta

El insumo concreto. el supervised fine-tuning (SFT) entrena sobre pares (instrucción → respuesta ideal), no sobre texto crudo. Un dataset de instruction tuning es una colección de miles de esos pares. «Instruction tuning» es el nombre del SFT aplicado a este tipo de pares.

Qué hay en un par. la instrucción es lo que un usuario pediría («Resumí esto», «Traducí al inglés»). La respuesta ideal es lo que un buen asistente contestaría: la demostración de «así se responde bien».

Por qué generaliza. la clave es la variedad. Con instrucciones de muchos tipos —resumir, traducir, explicar, clasificar—, el modelo no memoriza respuestas puntuales: aprende la habilidad general «dada una instrucción, producí una respuesta útil». Por eso responde bien a instrucciones que nunca vio.

La idea. instruction tuning enseña la habilidad general de seguir instrucciones —no un catálogo de respuestas— con pares variados de instrucción y respuesta ideal.

resumir

«Resumí este texto.» → «...»

traducir

«Traducí al inglés: hola.» → «hello»

explicar

«¿Qué es una célula?» → «...»

clasificar

«¿Positivo o negativo? Me encantó.» → «positivo»

*la habilidad
general, no las
respuestas*

Cómo se ve un dataset de instrucciones

Cada ejemplo es un registro estructurado. instrucción, un contexto opcional (texto sobre el que operar) y la respuesta. Formato típico: JSON, un objeto por ejemplo.

Los abiertos que usamos. Dolly 15k (Databricks): 15.000 pares escritos por humanos, de uso comercial libre. OpenAssistant (OASST): árboles de conversación escritos y rankeados por la comunidad. (Alpaca —52.000 pares— es el famoso)

Calidad > cantidad. el hallazgo que ordena todo: LIMA (Less Is More for Alignment) mostró que mil ejemplos buenos y variados le ganan a un set enorme y ruidoso. Coherente con lo anterior: el SFT dirige lo que ya está, no enseña de cero — por eso alcanza con pocos, si son buenos.

La lección. pocos ejemplos, muy buenos y variados. En SFT, curar bien pesa más que acumular volumen.

un registro real

```
{  "instruction": "Resumí la fotosíntesis.",
  "context": "",
  "response": "Convierten luz en energía." }
```

pretraining · ~billones de tokens

SFT · ~1k–15k

dos o tres órdenes menos

categorías variadas:

resumen

QA

generación

clasificación

brainstorming

El mismo loop, pero solo se aprende la respuesta

Mecánicamente, no hay nada nuevo. el par se concatena en un stream de tokens: [tokens de la instrucción][tokens de la respuesta]. Y sobre eso, el loop de siempre: forward → loss → backward → update, predecir el próximo token.

La sutileza clave: se enmascara el prompt. la loss se calcula solo sobre los tokens de la respuesta, no los de la instrucción. La instrucción se pasa como contexto (la condición), pero no queremos que el modelo aprenda a generarla: queremos que produzca la respuesta dada la instrucción.

Por qué importa. sin enmascarar, el modelo también aprendería a inventar instrucciones —gastando capacidad en algo que no le pedimos y diluyendo la señal—. Enmascarar concentra el aprendizaje en el comportamiento que buscamos: responder.

La regla. la instrucción condiona, la respuesta enseña. El modelo ve todo el stream, pero la loss solo mira la respuesta.

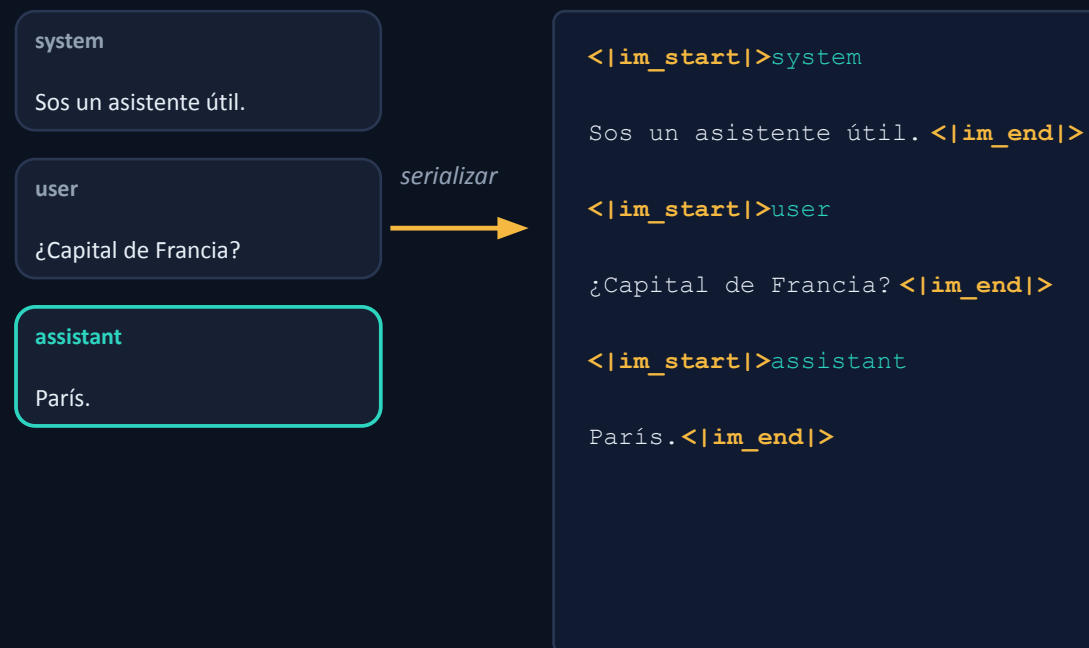


De conversación con roles a un stream de tokens

El problema. un asistente real no es un solo par: es una conversación multi-turno con roles (quién habla) y turnos que se alternan. Pero el modelo solo ve un stream plano de tokens. ¿Cómo se codifica esa estructura?

La solución: un chat template. una convención fija que serializa la conversación en un único stream, marcando roles y límites de turno con tokens especiales — los mismos tokens especiales que ya vimos al tokenizar: marcadores estructurales que no son texto, sino señales que el modelo aprende a reconocer.

Los tres roles. system (las instrucciones y la persona del asistente: «Sos un asistente útil y conciso»), user (los mensajes del humano) y assistant (las respuestas del modelo).



ChatML · tokens especiales en ámbar

La idea. el chat template traduce la estructura (roles, turnos) a tokens especiales, para que una conversación quepa en el único formato que el modelo entiende: un stream de tokens.

Entrenás y servís con el mismo template

El modelo aprende el template en el SFT. los datos de entrenamiento vienen formateados con el template, así que el modelo asocia los tokens especiales con los roles y aprende algo concreto: después del marcador de assistant, viene una respuesta útil.

En inferencia hay que usar el mismo. formateás el mensaje del usuario con los mismos tokens especiales, terminás justo en el marcador de assistant, y el modelo continúa como asistente. Cada modelo trae su propio template.

Por qué importa (bug clásico). si le hablás con otro template —o con texto crudo— el modelo se comporta peor: fue entrenado para responder dentro de un formato; si lo rompés, rompés el comportamiento. En HuggingFace esto se resuelve solo: `tokenizer.apply_chat_template(messages)` aplica el template exacto de ese modelo.

La regla. mismo template en entrenamiento y en inferencia. Romperlo rompe el comportamiento — por eso `apply_chat_template`.

```
messages [system, user]
```

```
apply_chat_template()
```

```
<|im_start|>user  
¿Capital de Francia?<|im_end|>  
  
<|im_start|>assistant
```

← termina acá, abierto: el modelo arranca a generar

mismo template ✓

otro template ✗

De completador a asistente

El resultado. base + instruction tuning = un modelo instruct (o chat).
Re-corremos el ejemplo del arranque: ante «¿Cuál es la capital de Francia?», el modelo instruct ahora responde «París.» — deja de completar el patrón y contesta.

Qué cambió, y qué no. es el mismo modelo base con los pesos ajustados: el conocimiento no cambió, el comportamiento sí. No le enseñamos nada nuevo del mundo; dirigimos lo que ya estaba.

Mismos pesos, ajustados. el instruct no es un modelo nuevo: es el base con los pesos movidos por el SFT. Vale la pena mirar de cerca ese ajuste — porque es, casi, el mismo entrenamiento del preentrenamiento.

prompt	modelo base	modelo instruct
«¿Capital de Francia?»	«¿Y la de Alemania?...» ✗	«París.» ✓
«Resumí: {texto}»	copia / divaga ✗	un resumen breve ✓

mismos pesos, ajustados — mismo conocimiento, otro comportamiento

El cierre de la idea. instruction tuning convierte el completador de texto en un asistente: mismo conocimiento, comportamiento nuevo.

El mismo entrenamiento, tres diferencias

Idéntico: el mecanismo. un SFT básico —lo que se llama full fine-tuning— tiene todos los pesos entrenables y los actualiza a todos. Mismo objetivo (predecir el próximo token, misma cross-entropy), el mismo loop de entrenamiento de siempre (forward → loss → backward → update), el mismo Transformer sin capas nuevas. Es, tal cual, el entrenamiento del preentrenamiento.

Distinto: tres cosas. pero no es idéntico en todo. Tres diferencias lo separan del preentrenamiento de cero — y las tres explican por qué el SFT dirige en vez de re-enseñar.

IGUAL: el loop ↔ CAMBIA: estas 3

1

De dónde arranca. Del checkpoint del modelo base, no de pesos random (igual que el continued pretraining). No entrena de cero: continúa lo ya entrenado.

2

La loss se enmascara. En pretraining cae sobre todos los tokens; en SFT, solo sobre la respuesta (el prompt masking de la lámina anterior). Diferencia real en el gradiente, aunque el loop sea el mismo.

3

Escala y learning rate. Pocos pasos (1–3 epochs sobre miles de ejemplos, no billones de tokens) y LR mucho más chico ($\sim 1e-5$). Empujar fuerte pisa lo que compró el pretraining — catastrophic forgetting. Los pesos se mueven apenas.

El pivote a lo que viene. es el mismo entrenamiento — pero mueve todos los pesos. A escala real, eso es caro. La próxima sección: cómo ajustar el comportamiento sin re-tocar todo el modelo.

Este procedimiento de SFT es el estándar de la industria

No es una maqueta didáctica. SFT full fine-tuning sobre pares instrucción→respuesta, con reward model y PPO atrás —lo que armamos en esta sección— es, tal cual, el pipeline que usan las empresas para llevar un modelo preentrenado a un asistente.

Lo cerrado es la receta puntual, no el método. Qué datasets exactos, cuántos ejemplos, qué mezcla de tareas, qué hiperparámetros finos — eso cada compañía lo guarda para sí. Es el mismo patrón que vimos con la curación de datos de preentrenamiento: el esquema general es conocido, la receta particular no.

Pero las bases sí están publicadas. El paper de InstructGPT (Ouyang et al., 2022) documenta el pipeline de tres pasos —SFT → reward model → PPO— que es la base directa de GPT-3.5. No es un secreto: es la referencia abierta sobre la que construimos toda esta sección.

LO PÚBLICO

- ✓ SFT (demostraciones)
- ✓ Reward model (preferencias)
- ✓ PPO (RLHF) — InstructGPT, 2022

↑ es lo que acabás de construir

LO CERRADO

- ✗ Dataset exacto
- ✗ Mezcla de tareas
- ✗ Hiperparámetros finos

La base está dada. Lo que sigue no cambia el método — cambia cuánto cuesta aplicarlo.

El costo es el único obstáculo

El método funciona. El problema es quién puede pagarlo. Full fine-tuning mueve todos los pesos del modelo. A la escala de un GPT-3.5, eso es cómputo que solo un puñado de empresas tiene.

Lo que sigue no es otro método — es el mismo, accesible. PEFT, LoRA y QLoRA no reemplazan al SFT que ya conocés: son formas de aplicar exactamente ese mismo mecanismo moviendo una fracción de los pesos, para que entre en un presupuesto de comunidad y no de datacenter.

El objetivo de lo que viene. No aprendés un método nuevo — aprendés a hacer el mismo SFT con menos plata. Por eso entra en una laptop lo que antes necesitaba un datacenter.



mismo mecanismo, otra escala de costo

FINE-TUNING

Ajuste eficiente

De mover todos los pesos a mover una fracción — el mismo comportamiento, mucho más barato.



Entrenar todo el modelo no entra en una GPU

Entrenar pide mucho más que los pesos. para correr el loop sobre todos los parámetros, en memoria conviven tres cosas por cada peso: el peso, su gradiente y los estados del optimizador. Con Adam —el que usamos— cada parámetro arrastra dos momentos extra.

La cuenta, en bytes por parámetro. sumando todo (pesos + gradiente + los dos estados de Adam + una copia de precisión) da del orden de 16 bytes por parámetro. Para un modelo de 7.000 millones, son ~112 GB — sin contar las activaciones.

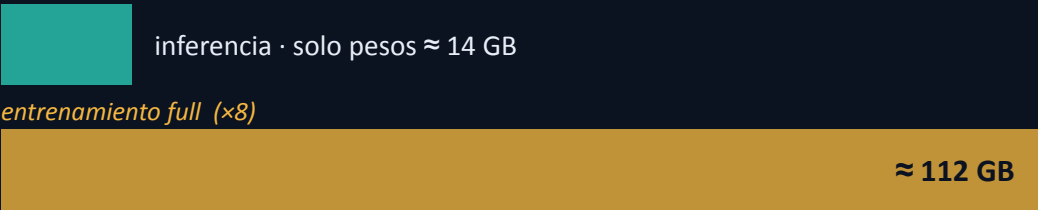
No entra, y encima se multiplica. 112 GB no entran en una GPU típica (una A100 tiene 80 GB; una T4, 16 GB). Y si hacés full fine-tuning para varias tareas, guardás una copia entera del modelo por cada tarea.

bytes por parámetro (entrenamiento con Adam)



los estados de Adam son la mitad del costo

memoria de un modelo de 7B



no entra en una GPU típica — T4: 16 GB · A100: 80 GB

El problema, en una línea. full fine-tuning es capaz, pero para modelos de verdad no entra en una sola GPU — y cada tarea se lleva otra copia completa.

Congelar el modelo y entrenar un puñado

La apuesta. PEFT (Parameter-Efficient Fine-Tuning) es la familia de métodos que congela el modelo base y entrena solo un conjunto muy chico de parámetros —nuevos o seleccionados—. El grueso del modelo no se toca.

Por qué alcanza. es la misma intuición de recién: el SFT dirige, no re-enseña. El conocimiento ya está en los pesos preentrenados; para cambiar el comportamiento no hace falta reescribir el modelo, alcanza con un ajuste chico. Si el cambio necesario es chico, no necesitás mover millones de pesos para lograrlo.

Qué se ahorra. y acá está la ganancia: los parámetros congelados no tienen gradiente ni estados de optimizador. Esos 14 bytes por parámetro se evaporan para casi todo el modelo; solo el puñado entrenable los paga. La memoria de entrenamiento se desploma hasta acercarse a la de inferencia.

Mismo SFT, otra economía. mismos pares instrucción→respuesta, misma loss enmascarada. Lo único que cambia es qué se mueve — por eso baja el costo, no la tarea.



Varias estrategias, un mismo principio

Todas congelan el base. PEFT no es un método sino una familia. Lo que comparten es la receta de recién: el modelo base queda congelado y lo entrenable es una fracción mínima. En lo que difieren es en dónde inyectan esa poca capacidad nueva.

Tres lugares donde ubicarlo.

- 1 - En capas nuevas intercaladas en el Transformer — adapters.
- 2 - En la entrada, como vectores aprendidos que se anteponen a la secuencia — soft prompts / prefix tuning.
- 3- Sobre las matrices de peso que ya existen, como un ajuste de bajo rango — LoRA.

Cómo seguimos. miramos primero adapters y soft prompts —qué son y qué cuestan— y después LoRA en detalle, que es el que vamos a usar en el lab.

El eje que las separa. no si se congela el base —eso lo hacen todas— sino dónde ponés lo poco que sí entrenás.

todas: base congelado · entrenable < 1%

Adapters

capas chicas entre las del Transformer

Soft prompts / prefix tuning

vectores aprendidos en la entrada

LoRA

un delta sobre los pesos que ya existen

[en detalle](#)

Capas insertadas, o vectores en la entrada

ADAPTERS

Módulos chicos insertados. entre las capas del Transformer se intercalan bloques nuevos y pequeños. Cada uno es un cuello de botella: baja la representación a una dimensión mucho menor, aplica una no linealidad y la vuelve a subir al tamaño original. Solo esos módulos se entrenan.

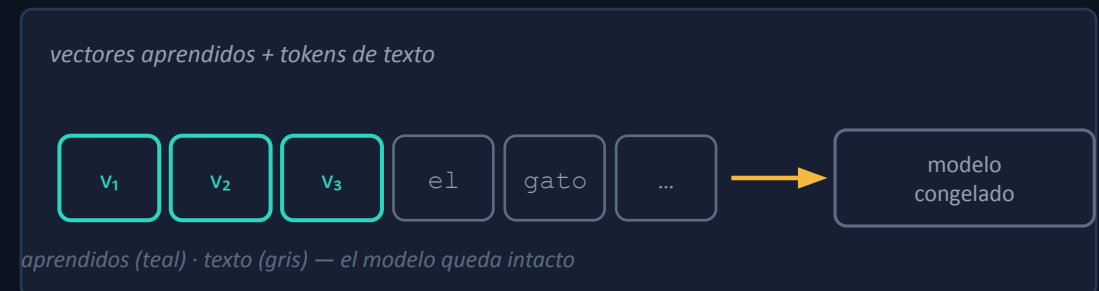
Qué cuesta. son capas reales agregadas al camino del forward: suman un poco de cómputo en cada inferencia — latencia extra mientras el adapter esté puesto.



SOFT PROMPTS / PREFIX TUNING

Aprender el preámbulo, no escribirlo. en vez de buscar a mano el prompt de texto ideal, se aprenden directamente unos vectores continuos que se anteponen a la entrada. No son palabras: son embeddings entrenables que el modelo lee como un preámbulo. Ningún peso se toca.

Prefix tuning. la variante que lleva esos vectores a todas las capas de atención —no solo a la entrada—: más capacidad, la misma idea de solo vectores, cero pesos.



Dos formas de no tocar los pesos. adapters agregan capas; soft prompts agregan vectores. En ambos, el modelo original queda congelado — cambia solo lo poco que se le suma alrededor.

Por qué LoRA se volvió el default

LoRA en una línea. el tercer lugar: en vez de agregar capas o vectores, aprende un ajuste de bajo rango sobre las matrices de peso que el modelo ya tiene. La próxima lámina abre esa idea.

Por qué ganó. junta tres cosas: es efectivo, es simple, y —la clave— su ajuste se puede fusionar con el peso original. Fusionado, en inferencia no agrega ni una sola operación.

El contraste que ya vimos. los adapters suman capas → algo de latencia siempre. Los soft prompts son más difíciles de optimizar y gastan parte del contexto. LoRA evita las dos cosas.

De acá en más, LoRA. es hoy el método por defecto para PEFT; el resto de la sección lo mira en detalle.

overhead en inferencia

Adapters

+ capas (latencia)

Soft prompts

gastan contexto

LoRA (fusionado)

estándar de facto

cero

El ajuste vive en pocas dimensiones

La observación. LoRA (Low-Rank Adaptation) parte de un hallazgo empírico: el ajuste que el fine-tuning le hace a una matriz de peso —el ΔW , cuánto cambió cada peso— resulta tener rango bajo. Aunque W sea enorme, el cambio se puede describir con muy poca información.

Qué significa rango bajo. que ΔW , en vez de ser una matriz llena e independiente en cada dirección, se puede aproximar bien como el producto de dos matrices flacas. Toda la matriz de cambio se reconstruye a partir de esas dos piezas chicas.

La movida. entonces no aprendamos ΔW entero. Aprendamos directamente esas dos matrices flacas cuyo producto lo aproxima. Muchísimos menos números que entrenar, para el mismo efecto.

La idea de LoRA. no entrenamos el cambio completo: entrenamos una versión flaca del cambio — dos matrices de rango r .

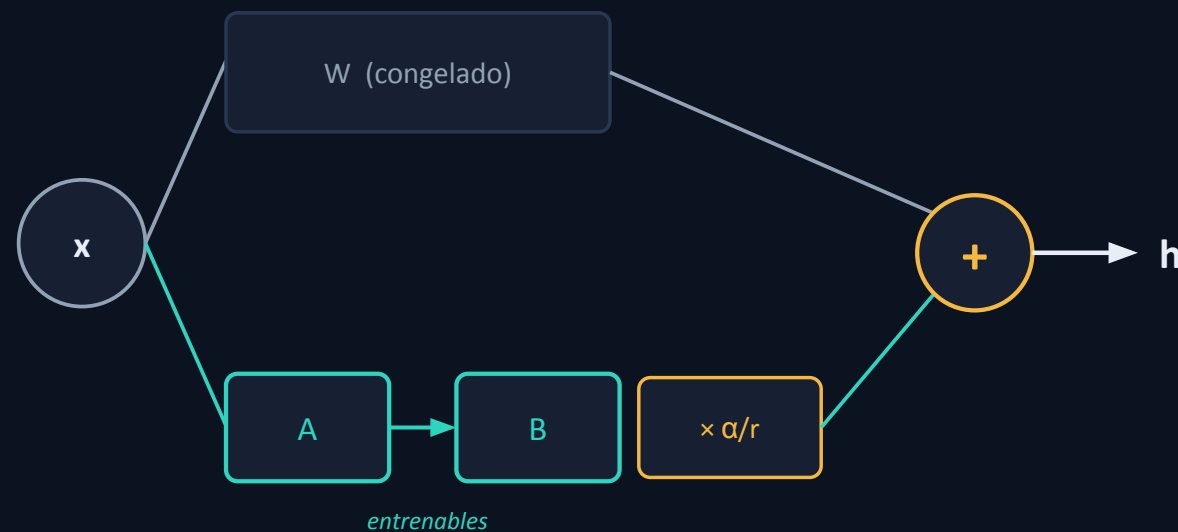


Dos caminos en paralelo al peso congelado

La forma. se deja W congelado y se le agrega un camino nuevo: $\Delta W = B \cdot A$, con A de tamaño $r \times d$ y B de $d \times r$, donde r es chico (típico: 8 o 16). Solo A y B se entrenan.

El forward. la salida es la suma de los dos caminos. El primer término es el modelo original intacto; el segundo es el ajuste LoRA. La entrada pasa por los dos y los resultados se suman.

Arranque y escala. A se inicializa al azar y B en cero $\rightarrow$ al empezar, $\Delta W = 0$: el modelo arranca idéntico al base, sin perturbarlo, y de ahí aprende. El factor α/r gradúa cuánto pesa el camino LoRA.



$$h = W \cdot x + (\alpha/r) \cdot B \cdot A \cdot x$$

El corazón de LoRA. dos matrices flacas en paralelo al peso congelado; el gradiente solo toca A y B .

Menos del 1% de los pesos

Una capa concreta. una proyección de atención en un modelo de dimensión 4096 es una matriz $4096 \times 4096 = 16,7$ millones de parámetros. En full fine-tuning, entrenás esos 16,7 M.

La misma capa con LoRA, $r=8$. A es 8×4096 y B es 4096×8 : juntas, $8 \times 4096 \times 2 = 65.536$ parámetros $\approx 0,4\%$ de esa capa. Doscientas cincuenta veces menos.

A escala del modelo entero. sumando sobre todas las capas donde se aplica LoRA, el total entrenable queda típicamente en $\sim 0,1-1\%$ del modelo. El otro 99%+ permanece congelado.

De dónde sale el ahorro. entrenás menos del 1% de los pesos — por eso baja tanto la memoria (menos gradientes y estados) y el disco (el adapter pesa poco).

parámetros entrenables · una capa 4096×4096

full fine-tuning

16,7 M

LoRA · $r = 8$

65,5 K

$\approx 0,4\%$

a la misma escala, la barra de LoRA casi no se ve

modelo completo → entrenable $\sim 0,1-1\%$

Dónde ponerlo y los pocos hiperparámetros

Dónde se inyecta. LoRA se aplica sobre matrices puntuales, no sobre todo el modelo. Lo más común: las proyecciones de atención (query, key, value, output); a veces también las de las capas MLP. Ese conjunto se llama los target modules.

Los hiperparámetros clave. son pocos: r (el rango del adapter — su capacidad), α (la escala del ajuste) y qué módulos target elegís. Nada más que eso define un setup de LoRA.

El trade-off. r más grande = más capacidad, pero más parámetros. En la práctica un r chico suele alcanzar —justamente porque el ajuste es de bajo rango—: se empieza chico y se sube solo si hace falta.

Poco que tunear. con el default —atención, $r = 8-16$ — suele andar bien; es un buen punto de partida.

bloque de atención



proyecciones donde se inyecta LoRA

r

8 – 16

rango / capacidad

α

16 – 32

escala del ajuste

target

q, k, v, o

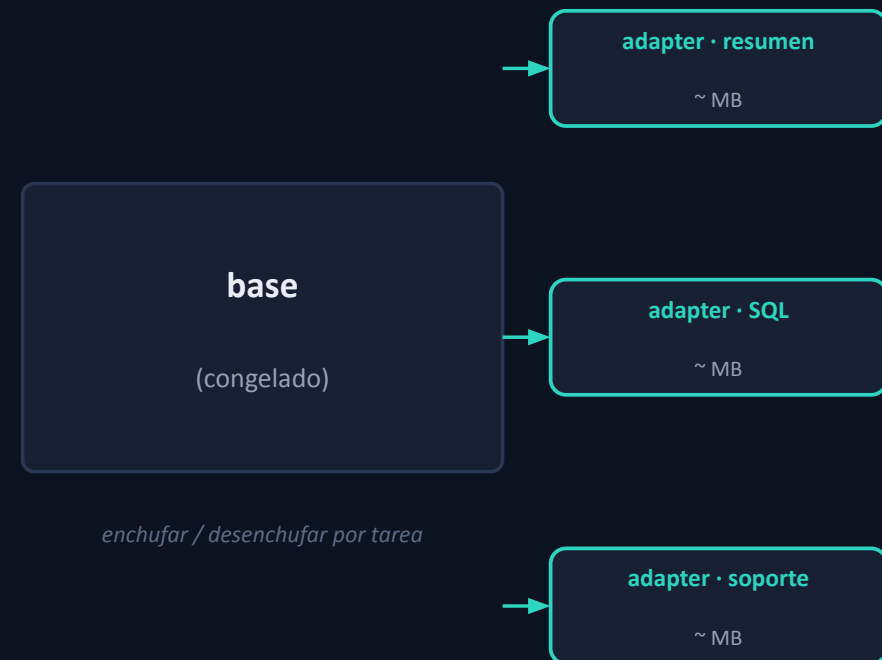
dónde se aplica

Poca memoria, y adapters intercambiables

Beneficio 1 — memoria mínima para entrenar. como solo A y B tienen gradiente y estados de optimizador, la memoria de entrenamiento se acerca a la de inferencia. Esto es lo que permite entrenar modelos grandes en GPUs chicas — y es la base sobre la que se monta QLoRA.

Beneficio 2 — adapters livianos e intercambiables. el adapter entrenado (A y B) pesa megabytes, no gigabytes. Un mismo base puede tener muchos adapters, uno por tarea, cargados y descargados en caliente; o fusionados al base para servir sin costo extra.

El contraste. frente al full fine-tuning —una copia entera del modelo por tarea— acá tenés un solo base y una carpeta de adapters chiquitos.



Un base, muchos comportamientos. cada comportamiento nuevo son unos megabytes — no otro modelo entero.

FINE-TUNING

QLoRA

El último paso de abaratamiento — entrenar con el modelo base comprimido a 4 bits.



Cuántos bits, y cómo se reparten

“Resolución” son dos preguntas. cuántos valores distintos podés representar —eso sale del total de bits: 2^{bits} — y qué tan fino es el paso y qué tan grande el rango —eso sale de cómo se reparten esos bits por dentro. No es un solo número.

Floating point reparte los bits. en fp16 y fp32, unos bits van al exponente (el rango: qué tan grande o chico) y otros a la mantisa (la precisión: cuántos dígitos). Como en notación científica $3,14 \times 10^8$: la mantisa (3,14) da precisión, el exponente (10^8) da escala.

Por eso hay dos “16 bits”. fp16 le da más a la mantisa (más preciso, rango chico); bf16 más al exponente (rango enorme, como fp32, menos preciso). Mismos 16 bits, distinto reparto — por eso bf16 aguanta mejor los gradientes al entrenar.

cuántos valores distintos · 2^{bits}

4 bits

$2^4 =$

16 valores

16 bits

$2^{16} =$

65.536 valores

32 bits

$2^{32} =$

~4.290 millones valores

NF4 usa esos 16 valores, puestos en los cuantiles de una normal — no es floating point.

los dos formatos de 16 bits · cómo reparten

fp16

5

10

bf16

8

7

exponente → rango

mantisa → precisión

La idea. más bits = más valores, siempre. Pero cómo se reparten esos bits (rango vs precisión) importa tanto como cuántos

El base congelado todavía ocupa memoria

Lo que LoRA resolvió. congelar el base eliminó los gradientes y los estados de optimizador de casi todo el modelo. La memoria de entrenamiento bajó muchísimo — de ahí vino todo el ahorro.

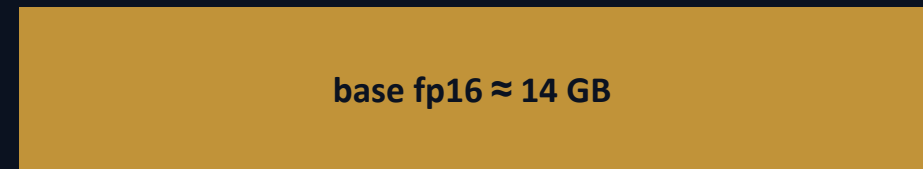
Lo que no resolvió. pero el base, aunque esté congelado, tiene que estar en memoria para el forward y el backward: la entrada tiene que pasar por sus pesos. Y esos pesos, en fp16, siguen pesando lo mismo — un modelo de 7B son ~14 GB solo de pesos.

Dónde aprieta. en una T4 de 16 GB, 14 GB de base dejan casi nada para las activaciones. Y para modelos más grandes (13B ≈ 26 GB, 65B ≈ 130 GB) el base ni siquiera entra en una sola GPU. El cuello dejó de ser el ajuste; ahora es el base.

El ajuste ya es barato; el base no. LoRA bajó todo lo entrenable a megabytes — pero seguís cargando 14 GB de pesos congelados.

memoria de entrenamiento · con LoRA

el cuello ahora



LoRA + gradientes ≈ MB

el base casi llena la GPU solo

Comprimir lo que no se entrena

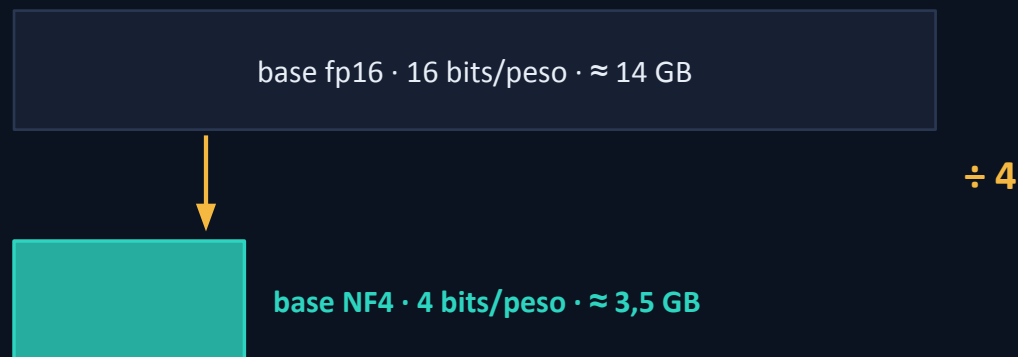
Qué es cuantizar. cuantización (quantization) es representar cada número con menos bits. Un peso en fp16 usa 16 bits; podemos guardarlo con solo 4, eligiendo —de un menú chico de valores— el más cercano. Menos precisión, cuatro veces menos memoria.

Por qué acá es seguro. el base está congelado: solo lo leemos en el forward, nunca lo actualizamos. Un peso que no se entrena no necesita precisión fina —no hay gradiente que se le aplique. Bajarle la precisión cuesta poco justamente porque no se mueve.

El cómputo no baja a 4 bits. al usar cada peso, se lo de-cuantiza al vuelo de vuelta a 16 bits, se hace la cuenta y se descarta. El almacenamiento es de 4 bits, pero la matemática sigue en 16. El ajuste LoRA sigue en alta precisión.

QLoRA = LoRA + base en 4 bits. mismo adapter entrenándose; lo nuevo es que el modelo congelado se guarda comprimido.

almacenamiento del base



en cada uso del peso, al vuelo:



se calcula y se descarta — no se guarda en 16 bits

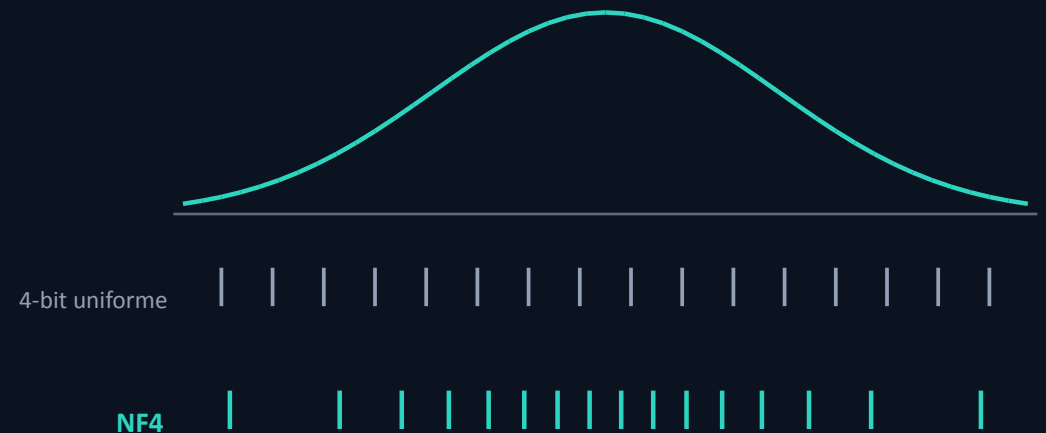
Un tipo de dato pensado para pesos

El problema de los 4 bits ingenuos. con 4 bits tenés 16 valores posibles, nada más. Si los repartís de forma uniforme en el rango, gastás resolución en zonas donde casi no hay pesos y te falta donde sí los hay.

Los pesos no son uniformes. los pesos de una red se distribuyen aproximadamente como una normal: muchísimos cerca de cero, pocos en los extremos. Un tipo de dato inteligente debería tener más valores donde los pesos se amontonan.

Eso es NF4. NF4 (4-bit NormalFloat) coloca sus 16 niveles en los cuantiles de una normal: densos cerca de cero, ralos en las colas. Mismos 4 bits, pero puestos donde están los pesos — así el error de redondeo es mucho menor.

los pesos se distribuyen $\sim$ normal



NF4 pone más niveles donde hay más pesos — menos error de redondeo

Cuatro bits, bien gastados. no es cuántos bits, sino dónde ponés los valores que esos bits pueden representar.

Afinar la memoria hasta el último bit

Double quantization. cuantizar no es gratis: cada bloque de pesos necesita una constante de escala para volver de los códigos de 4 bits a valores reales, y esas constantes ocupan memoria. Double quantization cuantiza también esas constantes — un ahorro de segundo orden, chico pero real.

Paged optimizers. durante el entrenamiento pueden aparecer picos de memoria puntuales (por ejemplo, secuencias largas) que harían reventar la GPU. Los paged optimizers descargan temporalmente los estados del optimizador a la RAM de la CPU cuando hay un pico, y los traen de vuelta después — como la paginación de un sistema operativo. Evitan el crash por out-of-memory.

Lo que suman. ninguno es la idea central —esa es cuantizar el base—, pero juntos hacen que QLoRA sea robusto y quepa de verdad sin sobresaltos.

Detalles que cierran la cuenta. no cambian la idea, pero son la diferencia entre "casi entra" y "entra y no se cae".

double quantization

las constantes de escala también ocupan memoria → se cuantizan también ellas.

constantes 32-bit



también 4-bit

paged optimizers

ante un pico de memoria, los estados del optimizador pasan a la RAM de la CPU y vuelven después.

GPU



RAM CPU

evita el out-of-memory

Un modelo grande, una sola GPU

Lo que habilitó. QLoRA hizo posible el fine-tuning de un modelo de 65B en una sola GPU de 48 GB — algo que en full fine-tuning pedía cientos de GB y varias GPUs. Y un 7B entra cómodo en una T4 de 16 GB, la que da Colab gratis.

Casi sin costo de calidad. el punto que sorprendió: entrenar con el base en 4 bits alcanza prácticamente la misma calidad que en 16 bits. NF4 + de-cuantización al vuelo + adapters en alta precisión recuperan lo que la compresión podría haber perdido.

El final de la escalera. full fine-tuning movía todo (caro) → LoRA movía una fracción (barato) → QLoRA hace eso mismo, pero además con el base comprimido (entra en cualquier lado). Es exactamente lo que vamos a usar en el lab.

De un cluster a un Colab gratis. el mismo SFT de siempre — la loss no cambió nunca; lo que cambió es cuánto hardware hace falta para correrlo.

la escalera completa

costo · hardware ↓

full fine-tuning todos los pesos · muchas GPUs



LoRA < 1% entrenable · GPU mediana



QLoRA < 1% + base 4-bit · GPU chica / gratis

7B fine-tuneable en una T4 gratis

FINE-TUNING

Síntesis y cierre

Un paso atrás para ver toda la escalera — y hacia dónde sigue.

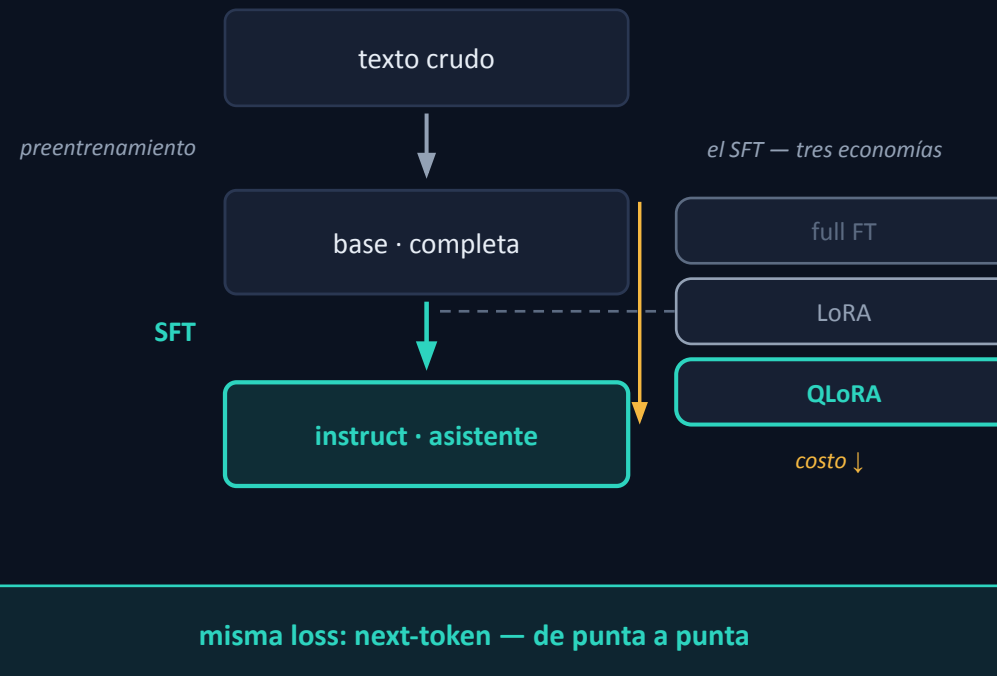


Una sola loss, cada vez más barato

Lo que nunca cambió: la loss. de punta a punta —preentrenamiento, SFT, LoRA, QLoRA— el objetivo fue siempre el mismo: predecir el próximo token. Nunca cambiamos qué optimiza el modelo; toda la clase se movió sobre una única función de pérdida.

Lo que cambió una vez: los datos. el salto de comportamiento fue un cambio de datos, no de objetivo: de texto crudo a demostraciones instrucción→respuesta. Ese solo cambio —qué le mostrás— convirtió al completador en asistente.

Lo que cambió en cada escalón: qué se mueve. el costo bajó por peldaños: todos los pesos (full fine-tuning) → una fracción mínima (LoRA) → esa fracción con el base a 4 bits (QLoRA). Cada paso, menos hardware para el mismo SFT.



Uno cambió el comportamiento, otro el costo — el tercero nunca se movió. datos nuevos te dan un asistente; mover menos pesos te lo da barato; la loss fue siempre la misma.

Clonar demostraciones tiene un techo

Lo que el SFT hace. el SFT enseña comportamiento clonando demostraciones: "acá tenés una buena respuesta, copιά este patrón". Funciona —transforma el base en asistente— pero en el fondo es imitación.

El techo de la imitación. el modelo puede llegar a ser tan bueno como sus demostraciones, no mejor. Y hay algo que la imitación no expresa: preferencias. Entre dos respuestas válidas, el SFT no tiene forma de decir "esta es mejor" — solo vio ejemplos sueltos para copiar, nunca comparaciones.

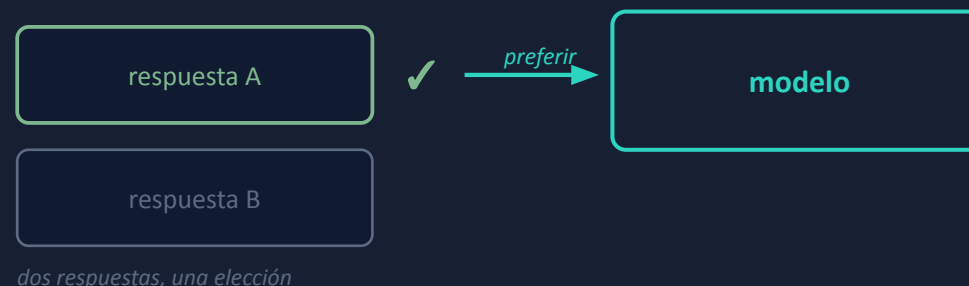
Lo que viene: aprender de preferencias. no reemplaza al SFT: viene después. Sobre el asistente ya formado, se le muestran pares de respuestas rankeadas por humanos y se lo optimiza hacia la preferida. Eso es RLHF (Reinforcement Learning from Human Feedback) y su versión directa, DPO.

De demostraciones a preferencias. el SFT te lleva lejos; el siguiente escalón abre la próxima etapa.

SFT · imitación



alineamiento · comparación



Todo esto, ahora en código

El lab. todo lo de hoy, en código y sobre una GPU gratis: cargar un modelo base chico, mostrar que no se comporta, aplicar el chat template, cuantizarlo a 4 bits y ponerle LoRA, imprimir qué fracción de los parámetros es entrenable, entrenar unos pasos, y comparar antes/después.

Lo que vas a ver con tus propios ojos. el $< 1\%$ de parámetros entrenables impreso en pantalla, un base de miles de millones de pesos entrando en una T4 de 16 GB, y el salto de completador a asistente en el mismo modelo — los números de estas láminas, ahora medidos en tu corrida.

el mapa del notebook



► **El mismo SFT de siempre — ahora en tu Colab.** la loss no cambió; lo que cambia es que esta vez lo corrés vos.

Referencias

PEFT — LOS MÉTODOS

- **Houlsby et al., 2019** — Parameter-Efficient Transfer Learning for NLP
- **Li & Liang, 2021** — Prefix-Tuning
- **Lester et al., 2021** — The Power of Scale for PE Prompt Tuning
- **Hu et al., 2021** — LoRA: Low-Rank Adaptation of LLMs
- **Dettmers et al., 2023** — QLoRA: Efficient Finetuning of Quantized LLMs

El corazón de esta clase. dos papers: LoRA (2021) y QLoRA (2023) — el resto es el mapa alrededor.

INSTRUCTION TUNING

- **Zhou et al., 2023** — LIMA: Less Is More for Alignment
- **Köpf et al., 2023** — OpenAssistant Conversations

ALINEAMIENTO — LO QUE VIENE

- **Ouyang et al., 2022** — Instruct-following with Human Feedback (InstructGPT)
- **Rafailov et al., 2023** — Direct Preference Optimization (DPO)

HERRAMIENTAS

- **HuggingFace** — PEFT + TRL (SFTTrainer)